

CodexRC — Documentación Maestra

De punta a punta: qué es, qué hace, cómo está armado y en qué versión está hoy

Generado el 06/10/2026 a partir del código fuente real del repositorio (README.md, ARCHITECTURE.md, AGENTS.md, CHANGELOG.md de 2222 líneas y los 102 módulos de `core/`). No contiene información inventada: cada dato sale de un archivo existente, citado donde corresponde.

1. Qué es CodexRC, en una frase

Motor de auditoría y caza de vulnerabilidades web, con backend local, evidencia determinista, veredictos falsables, y campañas masivas sobre plugins de WordPress y blancos autorizados. No es un escáner que dice "podría ser vulnerable": construye una cadena de evidencia (`SOURCE -> FLOW -> AUTH -> SANITIZATION -> SINK -> CORRELATION`) evaluada por "abogados" deterministas (nunca una IA que opina), y solo reporta cuando esa cadena está completa y es reproducible.

Las 5 reglas de oro (filosofía, de `README.md`)

1. **Sin evidencia no hay hallazgo.** Toda cadena se evalúa, nunca se infiere por intuición.
2. **Mejor 4 hallazgos que ejecutan de verdad que 20 que solo se reflejan.** VERITAS re-verifica cada candidato en navegador real con un canario inerte antes de sellarlo.
3. **El ruido muere en el motor, no a mano.** Guardas semánticas, refutación por dominancia y corpus de regresión eliminan falsos positivos solos, con registro del porqué.
4. **La tolerancia del edge no es vulnerabilidad.** Solo el *mismatch* observable edge/origin escala; la escalera de impacto nunca se salta (DETECTED sin reproducción no avanza).
5. **La caza respeta el blanco.** Sondas read-only A->B, canarios inertes, jitter, presupuestos de requests declarados por módulo, y parada al primer impacto confirmado. En vivo jamás se envenena la respuesta de un usuario tercero.

2. Arquitectura general (6 subsistemas)

Fuente: `ARCHITECTURE.md` . El layout físico es:

<code>backend/app.py</code>	Único entrypoint HTTP (Flask): rutas <code>/api/*</code> , jobs, watchdog, auth de Arsenal.
<code>core/*.py</code>	102 módulos, una responsabilidad cada uno (sin subcarpetas: organización por archivo plano).
<code>frontend/*.html</code>	3 UIs autocontenidas (index, hunter, arsenal).
<code>hechos/</code>	Memoria de "ya auditado" – NUNCA se vacía o se re-audita todo desde cero por error.
<code>reports/</code>	Informes JSON/PDF/TXT generados por job.
<code>docs/</code>	Especificaciones y esta documentación.

2.1 Motor de caza (corazón)

`hunter.py` es una **fachada** que re-exporta Spider, DeepHunter, XSSHunter, BlindXSS y XSSPro desde sus módulos `hunter_*.py` . Los consumidores (backend, CLI, `async_lane`) solo conocen la fachada, nunca los módulos internos directamente. `brain.py` , `self_tune.py` y `pipeline.py` orquestan decisiones.

2.2 Autenticación (auth.py , ~60KB)

Matrix universal: combina bases × rutas × payloads. La sesión heredada se guarda por origen (nunca pisa sesiones de otro blanco). Incluye `ghostgate.py` (evasión de Cloudflare vía navegador real, nombre fijado por el operador) y `waf_guard.py` (jitter + cooldown con memoria persistente).

2.3 Verificación anti-falsos-positivos (lo que hace confiable al motor)

Cadena de evidencia en orden: - `taint_trace.py` — TAIN-TRACE: sigue fuente->sink intra-archivo. - `gates_audit.py` — GATES-AUDIT: dictamina si un handler AJAX/REST está protegido (nonce/capability). - `evidence.py` — EVIDENCE-CHAIN: arma la cadena completa y la evalúa con jueces deterministas (nunca un LLM). - `fp_autoclose.py` — cierra falsos positivos ya conocidos. - `pattern_match.py` — CVE-MATCH con patrones ponderados contra una baseline histórica. - `veritas.py` — navegador real contra falsos positivos, con degradación elegante si no hay Chromium. - `observe.py` / `regress.py` — CODEX-OBSERVE / CODEX-REGRESS: el corpus de regresión (141 casos a la fecha de este documento). - `escalada.py` — sigue el hallazgo crítico con 4 pasos automáticos, solo sobre piezas verificadas.

2.4 Caza masiva (plugins de WordPress)

`wide_corpus.py` arma el universo (todo plugin de wordpress.org con =5.000 instalaciones). `hunt_wide.py` es el runner incremental (crash-safe, nunca re-audita lo ya hecho). `diff_hunt.py` audita solo código nuevo entre versiones. `plugin_batch.py`, `retro_hunt.py`, `vendor_farm.py` y `cve_matcher.py` completan la batería. `dossier_hunt.py` es el driver más reciente (Q4-2026), enfocado en las clases nuevas POI-REACH y MAGIC-CONFUSION.

2.5 Ingeniería inversa (binarios/APKs)

`bin_audit.py` (secretos en ELF), `re_engine.py` (parseo puro Python de DEX/JAR/ELF) y `decompile.py` (bytecode DEX -> pseudocódigo). Todo en Python puro para correr sin compilar en entornos como Termux/armv7l.

2.6 Backend + UI

Rutas Flask con jobs bajo lock, watchdog de procesos zombie (`zombies.py`). Seguridad: bind solo a loopback por defecto, logs sin secretos (passwords/tokens/cookies nunca se registran), Arsenal con modo EXTREMO que exige contraseña y token de desbloqueo temporal.

3. La escalera cero-falsos-positivos (cómo se decide un veredicto)

Principio transversal a todos los módulos nuevos (v0.85+): **cada peldaño exige el anterior**, nunca se salta un escalón. Ejemplo genérico de la familia desync/HTTP:

```
BENIGN -> X-DETECTED -> X-DESYNC -> X-STATE-EFFECT -> [HALLAZGO] SECURITY-IMPACT-DEMO
```

Y para Object Injection (`poi_reach.py`):

```
POI-DETECTED (unserialize con taint sin sanitizar)
-> POI-UNAUTH (el sink vive en superficie anónima)
-> POI-CHAIN (hay gadget con método mágico alcanzable)
-> POI-REACH ([HALLAZGO] candidato reportable)
```

Sin alcance anónimo, el veredicto honesto queda en **POI-AUTH**: es una pista, no un hallazgo reportable.

Solo dos símbolos están reservados para parar todo y avisar al operador: **[HALLAZGO]** (hallazgo explotable real) y **[ALERTA]/[CRITICO]** (hallazgo crítico). Todo lo demás es ruido que el motor filtra solo.

4. Línea de tiempo de versiones (resumen)

CodexRC lleva **78 versiones documentadas** en el CHANGELOG (desde v0.40 "Verificación del sistema" hasta v0.96.1). El detalle completo, versión por versión, está en [version_timeline.md](#) (generado automáticamente del CHANGELOG real). Hitos mayores:

Versión	Hito
v0.40 – v0.53	Cimientos: verificación, vendor-farm, plugin-batch, gates-audit, diff-hunt, evidence-chain, VDP-1300, UI-Renaissance
v0.57.x	AUTH matrix universal, auditoría de seguridad propia, triaje masivo de falsos positivos (BAC/SQLi)
v0.62.x	REVERSE-WEB, MULTI-IA (coordinación entre operadores), VDP-FRESH watcher cada 6h
v0.67 – v0.81	BAC-PROOF (IDOR dinámico en WP-LAB), ADAPTIVE-HUNT (capa epistémica), ARSENAL de 2 niveles
v0.82 – v0.84	Grafo de experimentos, selección por EDV, correlación cross-layer
v0.85 – v0.96.1	La corona (corona) : cadena completa de detección de desync HTTP (CL.0 -> H2 -> multi-conexión -> cross-request -> reproducción -> impacto -> FP-elimination -> evidence package), integración al Hunter (<code>crown_chain.py</code>), y el dossier Q4-2026 (POI-REACH, MAGIC-CONFUSION)

4.1 La corona (v0.85 -> v0.96.1), paso a paso

Esta es la cadena de detección de HTTP request smuggling / desync que se construyó módulo por módulo, cada uno con su propio corpus de regresión y validación en LAB antes de integrarse:

- v0.85 PIPE-CROSS** — nodo `cross_layer` en el pipeline de escaneo.
- v0.86 WCD-CACHE-KEY** — envenenamiento de caché vía clave mal normalizada.
- v0.87 CL.0-SINGLE-TIER** — smuggling de un solo salto.
- v0.88 H2-TRANSLATION** — edge HTTP/2 real traducido a H1 interno; oráculo de atribución por stream.
- v0.89 MULTI-CONNECTION** — una conexión envenena el pool y otra (TCP distinto) hereda el eco.
- v0.90 CROSS-REQUEST CORRELATION** — dos smuggles en la misma conexión, desplazamiento `k` medible y consistente.
- v0.91 REPRODUCTION ENGINE** — exige firma y `k` idénticos en dos pases independientes (A y B) antes de llamarlo reproducible.
- v0.92 IMPACT CORRELATION** — ¿el desync daña a un usuario *distinto* del atacante? Recurso simulado, atribución `ALIGNED / SMUGGLED-PROTECTED / MISPAIRED-BENIGN`.
- v0.93 FP-ELIMINATION** — batería de 4 cebos que nunca deben reportar + control positivo obligatorio.
- v0.94 EVIDENCE PACKAGE** — paquete falsable con hash SHA-256 canónico, comandos de reproducción y límites declarados.
- v0.95 INTEGRACIÓN AL HUNTER** — `crown_chain.py` orquesta los 5 peldaños (DETECTION -> REPRODUCTION -> STATE-EFFECT -> CROSS-CONNECTION -> SECURITY IMPACT) sin tocar los módulos

originales.

12. **v0.96.0 / v0.96.1 DOSSIER Q4-2026** — tres clases nuevas: POI-REACH (Object Injection con alcance), MAGIC-CONFUSION (confusión de extensión/contenido hacia Imagick), y el blindaje que cerró 2 falsos positivos reales encontrados en el primer barrido en vivo contra `weforms`.
-

5. Catálogo completo de módulos

Los 102 archivos de `core/` están indexados con su versión de origen y descripción real (extraída del encabezado de cada archivo) en [modulos_index.csv](#) — ábrilo en Excel/Sheets para filtrar por versión o buscar por palabra clave.

Agrupados por función (ver sección 2 para el detalle narrativo):

- **Detección de desync/HTTP:** `cl0_probe`, `h2_probe`, `mc_probe`, `crc_probe`, `edgesync`, `wcd_bait`, `wcd404_probe`, `cache_*` (bait, baseline, controls, correlation, fingerprint, semantic).
 - **Reproducción/impacto:** `repro_engine`, `race_proof`, `race_trace`, `chain`, `crown_chain`, `impact_probe`, `state_correlation`.
 - **Anti-FP / evidencia:** `fp_elimination`, `fp_autoclose`, `fp_memory`, `evidence`, `evidence_package`, `veritas`, `ghostgate`, `waf_guard`.
 - **Autenticación / IDOR / BAC:** `auth`, `bac_proof`, `gates_audit`, `poi_reach`, `normalization_audit`.
 - **Inyección:** `sqli_bait`, `ssti_bait`, `graphql_bait`, `magic_confusion`.
 - **XSS:** `hunter_xss`, `hunter_xsspro`, `hunter_blind`, `cspt_scan`.
 - **Caza masiva WordPress:** `wide_corpus`, `hunt_wide`, `diff_hunt`, `plugin_batch`, `dossier_hunt`, `retro_hunt`, `vendor_farm`, `cve_matcher`.
 - **Ingeniería inversa:** `bin_audit`, `re_engine`, `decompile`.
 - **Investigación/estrategia:** `research_memory`, `experiment_catalog`, `experiment_graph`, `experiment_selector`, `hypothesis_graph`, `adaptive_hunt`, `self_tune`, `brain`.
 - **Infraestructura/soporte:** `pipeline`, `zombies`, `cfg`, `connection_state`, `tech_detect`, `recon`, `taint_trace`, `report_export`, `vision_gate`, `semantic_*`.
-

6. Estado actual del motor (a la fecha de este documento)

- **Versión:** v0.96.1 "BLINDAJE POI-REACH".
 - **Corpus de regresión:** 141/141 casos, 0 fallos (certificado hoy).
 - **Auditoría de 588 plugins:** ejecutada completa (0 [HALLAZGO] confirmados, 6 candidatos POI-AUTH detectados). *Nota de transparencia: el JSON detallado con esos 6 candidatos se perdió en una limpieza de archivos temporales; el motor que los generó sigue intacto y puede re-correr el barrido cuando el operador dé la orden "LestGo".*
 - **Lo que el motor NO hace todavía de forma automática:** no dispara ataques de IDOR/lógica de negocio por sí solo. Hoy detecta candidatos (Fase 1: lectura estática de endpoints vía `gates_audit` / `poi_reach`) y valida BAC/IDOR dinámicamente contra un WP-LAB local (`bac_proof.py`, con usuarios Admin/Suscriptor/Anónimo), pero no arma automáticamente los payloads de lógica financiera (montos negativos, decimales truncados, sobreescritura de estado de pago). Esa pieza (el "Destructor de Lógica Financiera") es la que se definió construir a continuación.
-

7. Roadmap inmediato (decidido en esta conversación)

Objetivo: hacer que CodexRC sea mejor que Burp Suite Pro en control de acceso (IDOR) y lógica de negocio, su único punto débil real.

1. Ya tenemos: extracción estática de endpoints (`gates_audit / poi_reach`) + disparo multi-rol dinámico (`bac_proof.py` , Admin/Suscriptor/Anónimo contra WP-LAB).
 2. Falta: módulo de **inferencia de parámetros** (leer qué variables lee cada handler: `$_POST['ticket_id']` , `$_GET['user_id']` , etc.) y autocompletarlas con valores de prueba.
 3. Falta: **Destructor de Lógica Financiera** — detectar variables tipo `price / amount / qty / discount / total` y aplicarles la batería de ataques (negativos, decimales truncados, arrays, sobreescritura de estado `status=paid`).
-

8. Documentos relacionados

- [README.md](#) — filosofía, instalación, uso rápido.
- [ARCHITECTURE.md](#) — mapa técnico para editar sin romper.
- [AGENTS.md](#) — manual operativo para IAs que operan el repo.
- [CHANGELOG.md](#) — historial completo, versión por versión (2222 líneas).
- [version_timeline.md](#) — timeline condensado de las 78 versiones.
- [modulos_index.csv](#) — índice de los 102 módulos (abrir en Excel).
- [AUDITORIA_FALSOS_POSITIVOS.md](#) — metodología anti-FP detallada.
- [DOSSIER_INVESTIGACION_2026Q4.md](#) — investigación de las clases nuevas del Q4 2026.

Apendice - Indice completo de los 102 modulos

Modulo	Version	Descripcion
__init__		codexRC core package
ab_diff	v0.68.0	AB-DIFF universal (v0.68.0)
adaptive_hunt	v0.81.0	ADAPTIVE-HUNT (v0.81.0): la capa epistemica.
arsenal	v0.55.0	ARSENAL v0.55.0 — dos niveles: BASICO (verde, sin candado) y EXTREMO (rojo, contrasena).
async_lane	v0.23.0	SLIPSTREAM (carril async, v0.23.0)
auth		Authentication Module (Termux friendly)
bac_proof	v0.67.0	BAC-PROOF (v0.67.0, AUTHZ-PROOF capa 4)
bin_audit		BIN-AUDIT: analisis binario/nativo, mas alla del navegador.
brain		CEREBRO: la capa de inteligencia determinista del Hunter.
cache_bait		CACHE-BAIT: bateria dedicada de cache poisoning y cache deception.
cache_baseline	v0.79.0	CACHE-BASELINE (v0.79.0): clasifica el baseline del target.
cache_controls	v0.79.0	CACHE-CONTROLS (v0.79.0): explicaciones alternativas ANTES
cache_correlation	v0.79.0	CACHE-CORRELATION (v0.79.0): el orquestador.
cache_fingerprint	v0.79.0	CACHE-FINGERPRINT (v0.79.0): huella SOLO de senales
cache_semantic	v0.79.0	CACHE-SEMANTIC (v0.79.0): relacion entre Request A y Request B.
cfg	v0.58.1	SEMANTIC CORE (v0.58.1) — Nivel 1: CFG intra-funcion + dominancia.
chain		ENCADENAR HALLAZGOS: el valor real esta en las CADENAS, no en piezas.
cl0_probe	v0.87.0	v0.87.0 CL.0-SINGLE-TIER (Request Smuggling CL.0)
connection_state	v0.77.0	CONNECTION-STATE AUDIT (v0.77.0): ¿deja una peticion ambigua

cov_bait		COV-BAIT v2: caza guiada por COBERTURA DE CODIGO + MOTOR DE MUTACION.
crc_probe	v0.90.0	v0.90.0 CROSS-REQUEST CORRELATION
cross_layer	v0.84.0	v0.84.0 CROSS-LAYER CORRELATION
crown_chain	v0.95.0	v0.95.0 CROWN CHAIN — INTEGRACION AL HUNTER
cspt_scan	v0.71.0	CSPT-SCAN (v0.71.0): Client-Side Path Traversal.
cve_matcher		CVE Matcher Module
decompile	v0.36	DECOMPILE: descompilacion parcial nivel 3 (bytecode DEX -> pseudocodigo).
deep_scan		Deep Scan Module
desync_hunt	v0.83.0	DESYNC-HUNT (v0.83.0): investigacion DESYNC sobre el
diff_hunt	v0.45.0	DIFF-HUNT (v0.45.0)
dossier_hunt	v0.96.1	DOSSIER-HUNT (v0.96.1) — caza con las clases Q4-2026
edge_profile	v0.75.0	EDGE-PROFILE (v0.75.0): caracteriza el CONTRATO del edge.
edgesync	v0.74.0	EDGESYNC-HUNT (v0.74.0): SONDA-DE-CORRELACION.
escalada	v0.28	ESCALADA: que hacer DESPUES del aviso critico.
evidence	v0.47.0	EVIDENCE-CHAIN + ABOGADOS (v0.47.0)
evidence_package	v0.94.0	v0.94.0 EVIDENCE PACKAGE
experiment_catalog	v0.81.0	CATALOGO DE EXPERIMENTOS (v0.81.0): sondas HTTP de solo
experiment_graph	v0.83.0	EXPERIMENT-GRAPH (v0.83.0): grafo dirigido de
experiment_selector	v0.82	SELECTOR DE EXPERIMENTOS (v0.82): grafo de experimentos.
fp_autoclose	v0.44.0	FP-AUTO-CLOSE (v0.44.0)
fp_elimination	v0.93.0	v0.93.0 FP-ELIMINATION
fp_memory	v0.66.0	FP-MEMORIA (v0.66.0, AUTHZ-PROOF capa 5)

gates_audit	v0.43.0	GATES-AUDIT (v0.43.0)
ghostgate		Modulo GHOSTGATE (Cloudflare relay, Termux friendly)
graphql_bait		GRAPHQL: bateria dedicada contra endpoints GraphQL ocultos.
h2_probe	v0.88.0	v0.88.0 H2-TRANSLATION (Parser Differential)
hunt_wide	v0.48.0	v0.48.0 WIDE-HUNT: caza sobre el corpus COMPLETO wordpress.org (>=5k installs).
hunter	v0.57.5	Hunter (FACHADA, v0.57.5)
hunter_base		Motor de descubrimiento (arana + inventario de parametros) y corpus de
hunter_blind		from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hunter_deep	v0.57.2	Caza profunda multi-vector. Extraida de hunter.py (v0.57.2).
hunter_spider	v0.57.2	Arana de descubrimiento: mapa paginas/params/endpoints. Extraida de
hunter_xss		from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hunter_xsspro	v0.20.0	from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hypothesis_graph	v0.81.0	HYPOTHESIS-GRAPH (v0.81.0): hipotesis explicativas de una
impact_probe	v0.92.0	v0.92.0 IMPACT CORRELATION
lab_ab_site	v0.68.0	LAB-AB-SITE (v0.68.0): sitio NO-WordPress de prueba
magic_confusion	v0.96.0	MAGIC-CONFUSION (v0.96.0): extension vs contenido en rutas de media.
mc_probe	v0.89.0	v0.89.0 MULTI-CONNECTION (Efecto de Segundo Orden)
normalization_audit	v0.76.0	NORMALIZATION-AUDIT (v0.76.0): que representacion recibe el
observe		CODEX-OBSERVE (Fase 1, alcance reducido).
path_bait		PATH-BAIT: bateria dedicada de LFI / path traversal (PAGA en Patchstack).
pattern_match		CVE-MATCH: aprendizaje de patrones globales sobre codigo PHP.

pipeline		Pipeline Orchestrator
plugin_batch	v0.42.0	PLUGIN-BATCH (v0.42.0)
poi_reach	v0.96.1	POI-REACH (v0.96.1): Object Injection con alcance y gadget chain.
race_proof	v0.71.0	RACE-PROOF (v0.71.0): ejecutor dinamico de carreras TOCTOU.
race_trace	v0.70.0	RACE-TRACE (v0.70.0)
re_engine		REVERSE: ingenieria inversa nivel 2 (sobre BIN-AUDIT).
recon		Basic Recon Module
reg_bot	v0.68.0	REG-BOT (v0.68.0, AB-DIFF universal)
regress		CODEX-REGRESS (alcance reducido: Regression Corpus + ejecucion).
report_export		Reportes exportables del Hunter: TXT, JSON y PDF.
repro_engine	v0.91	REPRODUCTION ENGINE v0.91 — etapa 21 del roadmap
research_memory	v0.81.1	RESEARCH-MEMORY (v0.81.1): memoria de investigacion por
retro_hunt	v0.51.0	RETRO-HUNT (v0.51.0) — CodexRC
saml_audit	v0.71.0	SAML-DEFENSE (v0.71.0): auditor SAML/XML de PHP.
self_tune		AUTOCORRECCION: el Hunter recuerda que rindio y ajusta su propio gasto.
semantic_channels	v0.80	Canales de observacion de SEMANTIC-CACHE (v0.80).
semantic_core	v0.58.0	SEMANTIC CORE (v0.58.0) — Nivel 0: identidad canonica de recursos.
semantic_dimension	v0.80	Fase 2 de SEMANTIC-CACHE: localizacion de dimension y
semantic_engine	v0.80	SEMANTIC-CACHE (v0.80): motor de desacuerdos semanticos.
semantic_judge	v0.80	Juez determinista de SEMANTIC-CACHE (v0.80).
semantic_models	v0.80	Modelos de datos de SEMANTIC-CACHE (v0.80).
sqli_bait		SQLI-BAIT: deteccion avanzada de inyeccion SQL en la superficie de la app.

ssa		SSA-lite: grafo de versiones def-use para Taint-Trace.
ssti_bait		SSTI-BAIT: batería dedicada de inyección de plantillas con FINGERPRINT.
state		Estado individual por operador/IA.
state_correlation	v0.78.0	STATE-CORRELATION (v0.78.0): un veredicto por target, no por
stealth_bait		STEALTH-BAIT: caza de superficie oculta para blancos blindados.
taint_trace		TAINT-TRACE: análisis de flujo de datos (taint) sobre código fuente PHP.
tech_detect		Basic Technology Detection (html.parser)
universal_engine	v0.52.0	UNIVERSAL-ENGINE (v0.52.0) — CodexRC
vdp_fresh	v0.61.0	v0.61.0 VDP-FRESH: cazar los VDP RECIEN AGREGADOS a Patchstack.
vdp_watcher	v0.62.6	VDP-FRESH WATCHER (v0.62.6).
vendor_farm		VENDOR-FARM: descubre familias de plugins de UN MISMO vendor con
veritas		VERITAS: verificación de XSS con navegador real (Chrome headless).
vision_gate	v0.69.0	VISION-GATE (v0.69.0): clasificador visual de challenges via Gemini.
waf_guard	v0.24.0	GHOST-SHIELD (evasión WAF con memoria, v0.24.0)
wcd404_probe	v0.96.0	WCD-404-PROBE (v0.96.0): el oráculo de la cache que guarda 404s.
wcd_bait	v0.86.0	v0.86.0 WCD-CACHE-KEY (Web Cache Deception)
wide_corpus	v0.48.0	v0.48.0 WIDE-CORPUS: corpus de caza = TODOS los plugins wordpress.org
zombies		ZOMBIE-KILL — mata procesos zombies y duplicados del backend CodexRC.